

The Ultimate Web Development Handbook



Asynchronous JavaScript

JavaScript runs code **one line at a time** — it's **single-threaded**. This means only one task can happen at any moment. Still, JavaScript can do things like wait for a timer or handle user clicks without stopping everything else.

This is because JavaScript uses **asynchronous behavior** for certain tasks.

Synchronous Code

This is how JavaScript normally works — **top to bottom**.

```
console.log("A");  
console.log("B");  
console.log("C");  
  
// Output:  
// A  
// B  
// C
```

Each line waits for the previous one to finish. That's **synchronous** execution.

Asynchronous Code

Here's where it gets interesting:

```
console.log("A");  
  
setTimeout(() => {
```

```
    console.log("B");
  }, 1000);

  console.log("C");

  // Output:
  // A
  // C
  // B (after about 1 second)
```

Even though `setTimeout` is written before `"C"`, it runs **after**. Why?

What Happens Behind the Scenes

When JavaScript sees `setTimeout`, it:

1. Sends the task (along with its delay) to the **browser** (or **Node.js**, if you're running it there).
2. Continues running the rest of the code — it doesn't wait.
3. After the timer finishes, the function you passed to `setTimeout` is **sent back** to JavaScript to be run.
4. But it will only run **after** the current code is done.

This system of handling things is managed by the **event loop**.

The Event Loop (In Simple Words)

The **event loop** is a mechanism that:

- Keeps checking if JavaScript is done running your current code.
 - If yes, it picks up any pending tasks (like the `setTimeout` callback) and runs them.
-

Real-Life Analogy

Imagine you're cooking:

- You put rice on the stove and set a timer.
- You don't just stand there — you cut vegetables.
- When the timer rings, you go back to check the rice.

Similarly, JavaScript **schedules** tasks like timers to run **later**, and continues with the rest of the code.

Another Example

```
console.log("Start");

setTimeout(() => {
  console.log("Waiting over");
}, 2000);

console.log("End");

// Output:
// Start
// End
// Waiting over (after ~2 seconds)
```

Even with 2 seconds delay, "End" appears **immediately** after "Start" — that's the power of async.

Summary

- JavaScript is single-threaded: one thing at a time.
- Functions like `setTimeout` are **asynchronous**.
- These async tasks are handled by the browser/Node and returned later.

- The **event loop** checks when JavaScript is free to run those returned tasks.

Understanding this helps you write programs that don't get "stuck" waiting and can handle things like user input, network requests, and timers smoothly.

Introduction to JavaScript Promises

A **Promise** in JavaScript is a way to handle **asynchronous operations**. It lets you write code that runs **after something finishes**, without getting stuck in messy nested callbacks.

Think of a Promise like a **placeholder for a value** that will be available in the future.

Why Do We Need Promises?

With callbacks, things can quickly become hard to read and maintain, especially when we have to wait for multiple things.

Example of **callback hell**:

```
doTask1(function (result1) {  
  doTask2(result1, function (result2) {  
    doTask3(result2, function (result3) {  
      console.log("All tasks done");  
    });  
  });  
});
```

This kind of nested code becomes difficult to manage. **Promises solve this** by allowing a cleaner, more readable structure.

Basic Promise Syntax

```
const promise = new Promise(function (resolve, reject) {  
  // Do some work...
```

```
// Call resolve(result) if successful
// Call reject(error) if there's an error
});
```

Once a Promise is created, we can handle its result using `.then()` and `.catch()` :

```
promise
  .then(function (result) {
    // This runs if the promise was resolved
  })
  .catch(function (error) {
    // This runs if the promise was rejected
  });
```

A Simple Example: Fake Async Task

Let's create a Promise that waits for 2 seconds and then resolves.

```
function waitTwoSeconds() {
  return new Promise(function (resolve, reject) {
    setTimeout(function () {
      resolve("Done waiting");
    }, 2000);
  });
}

console.log("Start");

waitTwoSeconds()
  .then(function (message) {
    console.log(message); // "Done waiting"
  })
  .catch(function (error) {
    console.log("Something went wrong");
  });
```

```
console.log("End");
```

Output:

```
Start  
End  
Done waiting
```

Even though the Promise is written earlier, it runs **after** the rest of the synchronous code — just like with callbacks.

Solving Callback Hell with Promises

You can chain multiple `.then()` calls instead of nesting:

```
doTask1()  
  .then(function (result1) {  
    return doTask2(result1);  
  })  
  .then(function (result2) {  
    return doTask3(result2);  
  })  
  .then(function (result3) {  
    console.log("All tasks done");  
  })  
  .catch(function (error) {  
    console.log("Something failed", error);  
  });
```

This is **much cleaner** than deeply nested callbacks.

Summary

- A **Promise** is an object representing a value that may be available now, later, or never.
- It has three states:
 - **Pending**: not yet finished
 - **Resolved**: finished successfully
 - **Rejected**: finished with an error
- Use `.then()` to handle success and `.catch()` to handle errors.
- Promises help write **cleaner async code**, especially when chaining tasks.

JavaScript `async` and `await`

Writing asynchronous code using `.then()` and `.catch()` works well, but as your code grows, it can still feel a bit hard to follow.

JavaScript gives us a cleaner way to work with Promises:

`async` and `await`

What is `async` ?

If you put the keyword `async` before a function, it **automatically returns a Promise**.

```
async function greet() {  
  return "Hello";  
}  
  
greet().then(function (message) {  
  console.log(message); // "Hello"  
});
```

Even though we just returned a string, `greet()` becomes a Promise.

What is `await` ?

The `await` keyword is used **inside an `async` function**. It tells JavaScript to **wait for the Promise to resolve**, then continue.

Basic Example

Let's simulate a delay using `setTimeout` wrapped in a Promise:

```
function waitTwoSeconds() {  
  return new Promise(function (resolve) {  
    setTimeout(function () {  
      resolve("Waited for 2 seconds");  
    }, 2000);  
  });  
}  
  
async function runTask() {  
  console.log("Start");  
  
  const result = await waitTwoSeconds();  
  console.log(result);  
  
  console.log("End");  
}  
  
runTask();
```

Output:

```
Start  
Waited for 2 seconds  
End
```

Why Use `async` and `await` ?

Let's compare the same logic using `.then()` :

```
waitTwoSeconds().then(function (result) {  
  console.log(result);  
});
```

It works, but once you have **multiple async operations**, the `.then()` style gets harder to follow.

With `await`, your code looks more like regular, synchronous code — even though it's asynchronous.

Handling Errors with `try...catch`

If a Promise rejects, you can catch the error using `try...catch`.

```
function fakeTask(fail) {  
  return new Promise(function (resolve, reject) {  
    setTimeout(function () {  
      if (fail) {  
        reject("Something went wrong");  
      } else {  
        resolve("Task completed");  
      }  
    }, 1000);  
  });  
}  
  
async function run() {  
  try {  
    const result = await fakeTask(false);  
    console.log(result);  
  } catch (error) {  
    console.log("Caught error:", error);  
  }  
}
```

```
run();
```

Summary

- `async` makes a function return a Promise.
- `await` pauses inside the function until the Promise is done.
- You can use `try...catch` to handle errors just like synchronous code.
- `async / await` makes asynchronous code easier to **read** and **write**.

JavaScript `async` and `await`

Writing asynchronous code using `.then()` and `.catch()` works well, but as your code grows, it can still feel a bit hard to follow.

JavaScript gives us a cleaner way to work with Promises:

`async` and `await`

What is `async` ?

If you put the keyword `async` before a function, it **automatically returns a Promise**.

```
async function greet() {  
  return "Hello";  
}  
  
greet().then(function (message) {  
  console.log(message); // "Hello"  
});
```

Even though we just returned a string, `greet()` becomes a Promise.

What is `await` ?

The `await` keyword is used **inside an `async` function**. It tells JavaScript to **wait for the Promise to resolve**, then continue.

Basic Example

Let's simulate a delay using `setTimeout` wrapped in a Promise:

```
function waitTwoSeconds() {  
  return new Promise(function (resolve) {  
    setTimeout(function () {  
      resolve("Waited for 2 seconds");  
    }, 2000);  
  });  
}  
  
async function runTask() {  
  console.log("Start");  
  
  const result = await waitTwoSeconds();  
  console.log(result);  
  
  console.log("End");  
}  
  
runTask();
```

Output:

```
Start  
Waited for 2 seconds  
End
```

Why Use `async` and `await` ?

Let's compare the same logic using `.then()` :

```
waitTwoSeconds().then(function (result) {  
  console.log(result);  
});
```

It works, but once you have **multiple async operations**, the `.then()` style gets harder to follow.

With `await`, your code looks more like regular, synchronous code — even though it's asynchronous.

Handling Errors with `try...catch`

If a Promise rejects, you can catch the error using `try...catch`.

```
function fakeTask(fail) {  
  return new Promise(function (resolve, reject) {  
    setTimeout(function () {  
      if (fail) {  
        reject("Something went wrong");  
      } else {  
        resolve("Task completed");  
      }  
    }, 1000);  
  });  
}  
  
async function run() {  
  try {  
    const result = await fakeTask(false);  
    console.log(result);  
  } catch (error) {  
    console.log("Caught error:", error);  
  }  
}
```

```
run();
```

Summary

- `async` makes a function return a Promise.
- `await` pauses inside the function until the Promise is done.
- You can use `try...catch` to handle errors just like synchronous code.
- `async / await` makes asynchronous code easier to **read** and **write**.

JavaScript Callbacks

A **callback** is simply a **function passed as an argument to another function**, to be **called later**.

This might sound confusing at first, but once you see it in action, it becomes very easy to understand.

Why Do We Need Callbacks?

Sometimes, you don't want a function to run immediately.

Instead, you want it to **run later, when something else happens** — for example:

- When a timer finishes
- When a user clicks a button
- When some data is ready

Callbacks help us do that.

Basic Example of a Callback

```
function greet(name) {  
  console.log("Hello, " + name);  
}  
  
function processUser(callback) {  
  const userName = "Harry";  
  callback(userName);  
}
```

```
processUser(greet);
```

What's Happening Here?

- `greet` is a function that prints a greeting.
- `processUser` is another function that receives a function (callback) and **calls it later**.
- We pass `greet` as an argument to `processUser`.

So `processUser(greet)` ends up calling: `greet("Harry")`

Callbacks in Asynchronous Code

Callbacks are often used with async functions like `setTimeout`.

```
function showMessage() {  
  console.log("This runs after 2 seconds");  
}  
  
setTimeout(showMessage, 2000);  
  
console.log("This runs first");
```

Output:

```
This runs first  
This runs after 2 seconds
```

Even though `showMessage` is written before the timer, it **runs later** — after 2 seconds. That's because we passed it as a callback to `setTimeout`.

Writing Inline Callback Functions

Instead of defining the function first, we can also write it directly:

```
setTimeout(function () {  
  console.log("Hello after 1 second");  
}, 1000);
```

This is still a callback — just written directly where it's used.

Another Example with User Input (Browser Only)

If you're in a browser and use something like this:

```
document.getElementById("btn").addEventListener("click", function () {  
  console.log("Button clicked");  
});
```

The second argument to `addEventListener` is a callback. It runs **only when the user clicks** the button.

Summary

- A **callback** is just a function passed to another function.
- It can be used to run code **later**.
- Callbacks are commonly used in:
 - `setTimeout`
 - Event listeners
 - Many asynchronous operations

Callbacks are the foundation for working with asynchronous JavaScript. Once you're comfortable with them, you're ready to learn more advanced things like **Promises** and `async/await` .

Introduction to JavaScript Promises

A **Promise** in JavaScript is a way to handle **asynchronous operations**. It lets you write code that runs **after something finishes**, without getting stuck in messy nested callbacks.

Think of a Promise like a **placeholder for a value** that will be available in the future.

Why Do We Need Promises?

With callbacks, things can quickly become hard to read and maintain, especially when we have to wait for multiple things.

Example of **callback hell**:

```
doTask1(function (result1) {  
  doTask2(result1, function (result2) {  
    doTask3(result2, function (result3) {  
      console.log("All tasks done");  
    });  
  });  
});
```

This kind of nested code becomes difficult to manage. **Promises solve this** by allowing a cleaner, more readable structure.

Basic Promise Syntax

```
const promise = new Promise(function (resolve, reject) {  
  // Do some work...
```

```
// Call resolve(result) if successful
// Call reject(error) if there's an error
});
```

Once a Promise is created, we can handle its result using `.then()` and `.catch()` :

```
promise
  .then(function (result) {
    // This runs if the promise was resolved
  })
  .catch(function (error) {
    // This runs if the promise was rejected
  });
```

A Simple Example: Fake Async Task

Let's create a Promise that waits for 2 seconds and then resolves.

```
function waitTwoSeconds() {
  return new Promise(function (resolve, reject) {
    setTimeout(function () {
      resolve("Done waiting");
    }, 2000);
  });
}

console.log("Start");

waitTwoSeconds()
  .then(function (message) {
    console.log(message); // "Done waiting"
  })
  .catch(function (error) {
    console.log("Something went wrong");
  });
```

```
console.log("End");
```

Output:

```
Start  
End  
Done waiting
```

Even though the Promise is written earlier, it runs **after** the rest of the synchronous code — just like with callbacks.

Solving Callback Hell with Promises

You can chain multiple `.then()` calls instead of nesting:

```
doTask1()  
  .then(function (result1) {  
    return doTask2(result1);  
  })  
  .then(function (result2) {  
    return doTask3(result2);  
  })  
  .then(function (result3) {  
    console.log("All tasks done");  
  })  
  .catch(function (error) {  
    console.log("Something failed", error);  
  });
```

This is **much cleaner** than deeply nested callbacks.

Summary

- A **Promise** is an object representing a value that may be available now, later, or never.
- It has three states:
 - **Pending**: not yet finished
 - **Resolved**: finished successfully
 - **Rejected**: finished with an error
- Use `.then()` to handle success and `.catch()` to handle errors.
- Promises help write **cleaner async code**, especially when chaining tasks.

JavaScript `async` and `await`

Writing asynchronous code using `.then()` and `.catch()` works well, but as your code grows, it can still feel a bit hard to follow.

JavaScript gives us a cleaner way to work with Promises:

`async` and `await`

What is `async` ?

If you put the keyword `async` before a function, it **automatically returns a Promise**.

```
async function greet() {  
  return "Hello";  
}  
  
greet().then(function (message) {  
  console.log(message); // "Hello"  
});
```

Even though we just returned a string, `greet()` becomes a Promise.

What is `await` ?

The `await` keyword is used **inside an `async` function**. It tells JavaScript to **wait for the Promise to resolve**, then continue.

Basic Example

Let's simulate a delay using `setTimeout` wrapped in a Promise:

```
function waitTwoSeconds() {  
  return new Promise(function (resolve) {  
    setTimeout(function () {  
      resolve("Waited for 2 seconds");  
    }, 2000);  
  });  
}  
  
async function runTask() {  
  console.log("Start");  
  
  const result = await waitTwoSeconds();  
  console.log(result);  
  
  console.log("End");  
}  
  
runTask();
```

Output:

```
Start  
Waited for 2 seconds  
End
```

Why Use `async` and `await` ?

Let's compare the same logic using `.then()` :

```
waitTwoSeconds().then(function (result) {  
  console.log(result);  
});
```

It works, but once you have **multiple async operations**, the `.then()` style gets harder to follow.

With `await`, your code looks more like regular, synchronous code — even though it's asynchronous.

Handling Errors with `try...catch`

If a Promise rejects, you can catch the error using `try...catch`.

```
function fakeTask(fail) {  
  return new Promise(function (resolve, reject) {  
    setTimeout(function () {  
      if (fail) {  
        reject("Something went wrong");  
      } else {  
        resolve("Task completed");  
      }  
    }, 1000);  
  });  
}  
  
async function run() {  
  try {  
    const result = await fakeTask(false);  
    console.log(result);  
  } catch (error) {  
    console.log("Caught error:", error);  
  }  
}
```

```
run();
```

Summary

- `async` makes a function return a Promise.
- `await` pauses inside the function until the Promise is done.
- You can use `try...catch` to handle errors just like synchronous code.
- `async / await` makes asynchronous code easier to **read** and **write**.

Fetch API in JavaScript

The **Fetch API** provides a modern way to make HTTP requests in JavaScript. It returns a **Promise**, making it easier to handle asynchronous requests compared to older methods like `XMLHttpRequest` .

Basic Syntax

```
fetch(url, options)
  .then(response => {
    // handle response
  })
  .catch(error => {
    // handle error
  });
```

- `url` : The resource you want to fetch.
 - `options` : (Optional) An object that contains settings such as method, headers, and body.
-

Example: GET Request

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => {
    if (!response.ok) {
      throw new Error("Network response was not ok");
    }
    return response.json();
  })
```

```
.then(data => {  
  console.log(data);  
})  
.catch(error => {  
  console.error("Fetch error:", error);  
});
```

Example: POST Request

```
fetch("https://jsonplaceholder.typicode.com/posts", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({  
    title: "foo",  
    body: "bar",  
    userId: 1,  
  }),  
})  
.then(response => response.json())  
.then(data => {  
  console.log("Created:", data);  
})  
.catch(error => {  
  console.error("Error:", error);  
});
```

Using `async/await`

```
async function fetchData() {  
  try {  
    const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
```

```
if (!response.ok) {  
  throw new Error("Network response was not ok");  
}  
  
const data = await response.json();  
console.log(data);  
} catch (error) {  
  console.error("Fetch error:", error);  
}  
}  
  
fetchData();
```

Key Points

- `fetch` returns a Promise that resolves to a `Response` object.
- You need to call methods like `.json()`, `.text()`, or `.blob()` to read the response body.
- Errors only occur on **network failure** or if `response.ok` is false.
- Supports request configuration with methods, headers, and body.

Introduction to Express.js

What is Express.js?

Express.js is a fast, unopinionated, and minimalist web framework for Node.js. It simplifies the process of building web servers and APIs using JavaScript.

Instead of writing raw HTTP code in Node.js, Express gives us a higher-level set of tools to build robust backend applications quickly and efficiently.

Why Use Express.js Over Node.js Core Modules?

Raw Node.js:

- You need to manually parse requests and handle routes.
- No built-in support for things like middleware, form data, sessions, or routing.

Express.js:

- Built-in routing support.
 - Middleware support for handling requests and responses.
 - Easily serve static files.
 - Simplifies API and web app development.
-

Real-World Use Cases

Express is used in many production-grade applications, such as:

- REST APIs

- Web applications (with HTML templating)
- Backend for mobile and single-page apps
- Server-side rendering setups

Installing Express.js

Before using Express, make sure Node.js and npm are installed.

To install Express:

```
npm install express
```

Your First Express App

Here's a basic example of an Express server:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

- `express()` creates an Express app.
- `app.get()` defines a route handler.
- `app.listen()` starts the server.

Visit `http://localhost:3000` in your browser to test it.

Installing and Setting Up Express.js

Project Setup

Before we start writing code, let's set up a new project folder.

1. Create a new folder:

```
mkdir express-intro  
cd express-intro
```

1. Initialize a new Node.js project:

```
npm init -y
```

This creates a `package.json` file with default settings.

1. Install Express:

```
npm install express
```

You'll now see `express` listed in your `dependencies`.

Creating Your First Express Server

Create a file called `index.js`:

```
const express = require('express');  
const app = express();
```

```
// Define a basic GET route
app.get('/', (req, res) => {
  res.send('Welcome to Express.js!');
});

// Start the server
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Testing the Server

1. Run the server using:

```
node index.js
```

1. Open your browser and visit:

```
http://localhost:3000
```

You should see the message: `Welcome to Express.js!`

Understanding the Code

- `express()` initializes the app.
 - `app.get()` sets up a route for GET requests to the `/` path.
 - `res.send()` sends a simple text response.
 - `app.listen()` starts the server on the specified port.
-

Routing in Express.js

What is Routing?

Routing refers to how an application responds to client requests to specific paths (URLs) using HTTP methods such as GET, POST, PUT, DELETE, etc.

In Express.js, routes define the logic for what should happen when a user visits a particular URL.

Basic GET Route

```
app.get('/', (req, res) => {  
  res.send('Home Page');  
});
```

- This responds to a GET request to the root (/) URL.
 - `res.send()` sends a response back to the client.
-

Route Methods in Express

Express provides methods for all standard HTTP methods:

```
app.get('/about', (req, res) => {  
  res.send('About Page');  
});  
  
app.post('/contact', (req, res) => {  
  res.send('Contact form submitted');
```

```
});

app.put('/user/:id', (req, res) => {
  res.send(`Update user with ID ${req.params.id}`);
});

app.delete('/user/:id', (req, res) => {
  res.send(`Delete user with ID ${req.params.id}`);
});
```

Route Parameters

Route parameters are named segments of the URL prefixed with a colon (:). They allow you to capture values from the URL.

```
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID is ${userId}`);
});
```

Query Parameters

Query parameters are added to the URL after a ? and are accessible using `req.query` .

```
// URL: /search?term=node
app.get('/search', (req, res) => {
  const searchTerm = req.query.term;
  res.send(`You searched for ${searchTerm}`);
});
```

Summary

- Use `app.METHOD()` to define routes.
- Use `req.params` to get route parameters.
- Use `req.query` for query parameters.